

- ↳ We introduce a new abstraction of a process
 - ↳ Each thread has its own PC, private set of registers, but they share the same address space
 - ↳ During context switch, register state of thread 1 must be saved & register state of thread 2 restored.
- We use Thread control block (TCB) to store the state of each thread of a process.

↳ Previously, we had a single stack per process (such a process is single threaded process)

In multi-threaded processes, each thread runs independently & of course call into various routines. So, there will be one stack per thread. Such a stack is also called thread local storage.

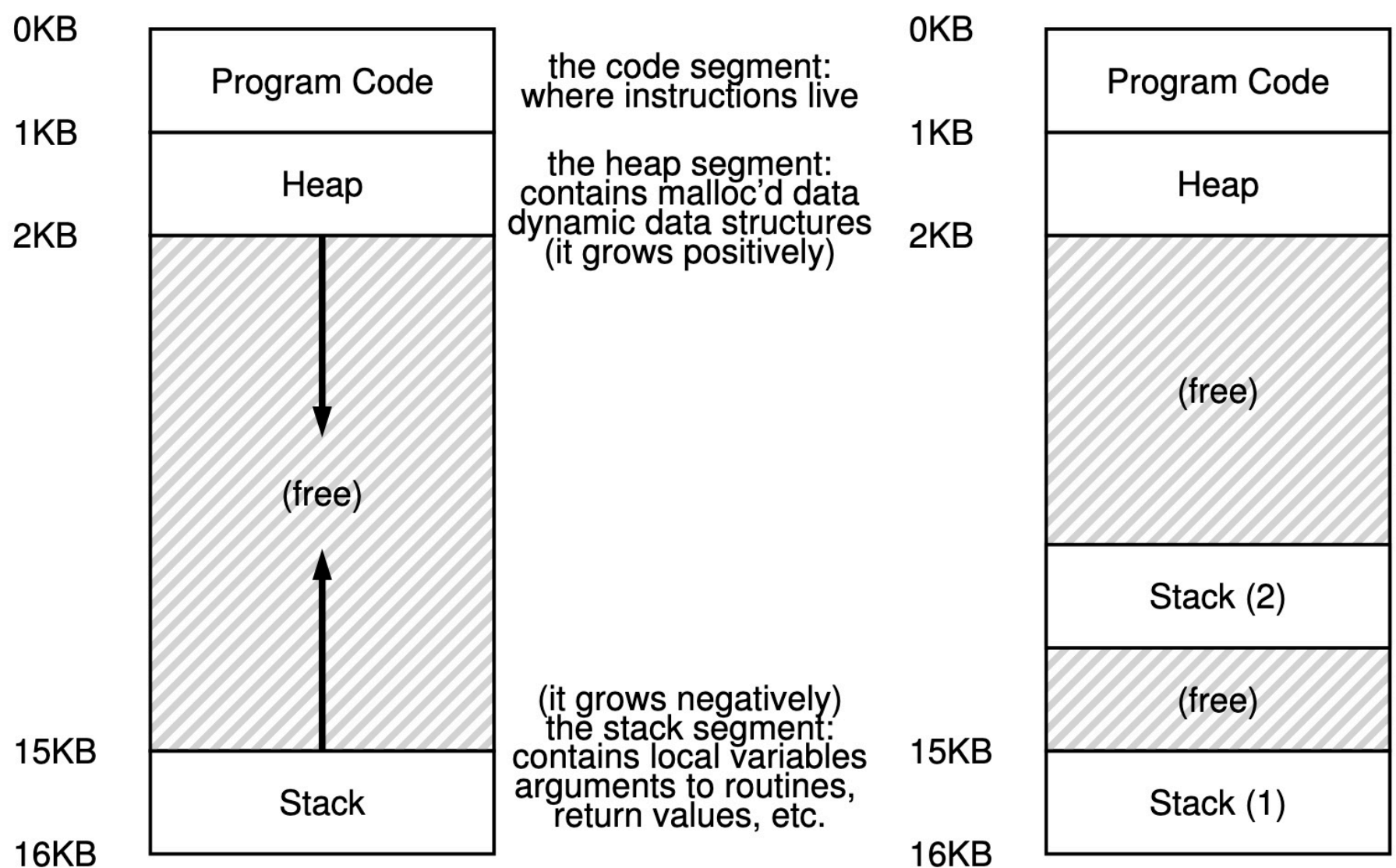


Figure 26.1: Single-Threaded And Multi-Threaded Address Spaces

Before, the stack and heap could grow independently and trouble only arose when you ran out of room in the address space. Here, we no longer have such a nice situation. Fortunately, this is usually OK, as stacks do not generally have to be very large (the exception being in programs that make

heavy use of recursion).

Why use Threads?

There are two major reasons:

1. Parallelism:

Imagine you are writing a program that performs operations on very large arrays, for example, adding two large arrays together, or incrementing the value of each element in the array by some amount. If you are running on just a single processor, the task is straightforward: just perform each operation and be done. However, if you are executing the program on a system with multiple processors, you have the potential of speeding up this process considerably by using the processors to each perform a portion of the work. The task of transforming your standard single-threaded program into a program that does this sort of work on multiple CPUs is called parallelization, and using a thread per CPU to do this work is a natural and typical way to make programs run faster on modern hardware.

2. To avoid blocking program progress due to slow I/O

Imagine that you are writing a program that performs different types of I/O: either waiting to send or receive a message, for an explicit disk I/O to complete, or even (implicitly) for a page fault to finish. Instead of waiting, your program may wish to do something else, including utilizing the CPU to perform computation, or even issuing further I/O requests. Using threads is a natural way to avoid getting stuck; while one thread in your program waits (i.e., is blocked waiting for I/O), the CPU scheduler can switch to other threads, which are ready to run and do something useful. Threading enables overlap of I/O with other activities within a single program, much like multiprocessing did for processes across programs; as a result, many modern server-based applications (web servers, database management systems, and the like) make use of threads in their implementations

But, for both cases, you can use multiple processes instead of threads. However, threads share an address space & make it easy to share data.

An Example: Thread Creation

Let us want to run a program that creates two threads, each of which does some independent work, say printing A & B.

```

1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4  #include "common.h"
5  #include "common_threads.h"
6
7  void *mythread(void *arg) {
8      printf("%s\n", (char *) arg);
9      return NULL;
10 }
11
12 int
13 main(int argc, char *argv[]) {
14     pthread_t p1, p2;
15     int rc;
16     printf("main: begin\n");
17     Pthread_create(&p1, NULL, mythread, "A");
18     Pthread_create(&p2, NULL, mythread, "B");
19     // join waits for the threads to finish
20     Pthread_join(p1, NULL);
21     Pthread_join(p2, NULL);
22     printf("main: end\n");
23     return 0;
24 }

```

Figure 26.2: Simple Thread Creation Code (t0.c)

- ↳ Once a thread is created, it may start running or be put in "ready" state (depends on scheduler).
 - ↳ After creating two threads (T_1 & T_2), the main thread calls `pthread_join()`, which waits for a particular thread to complete.
 - ↳ Overall, three threads were used: main thread, T_1 & T_2 .
- Below figure is possible execution order of the above program.

main	Thread 1	Thread2
starts running		
prints "main: begin"		
creates Thread 1		
creates Thread 2		

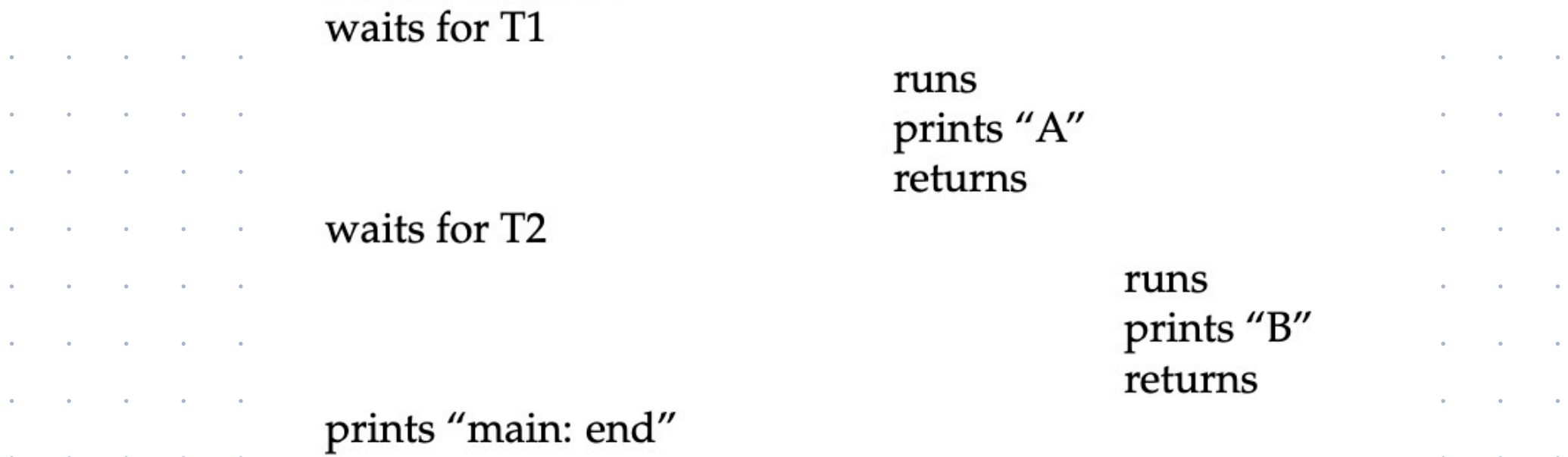


Figure 26.3: Thread Trace (1)

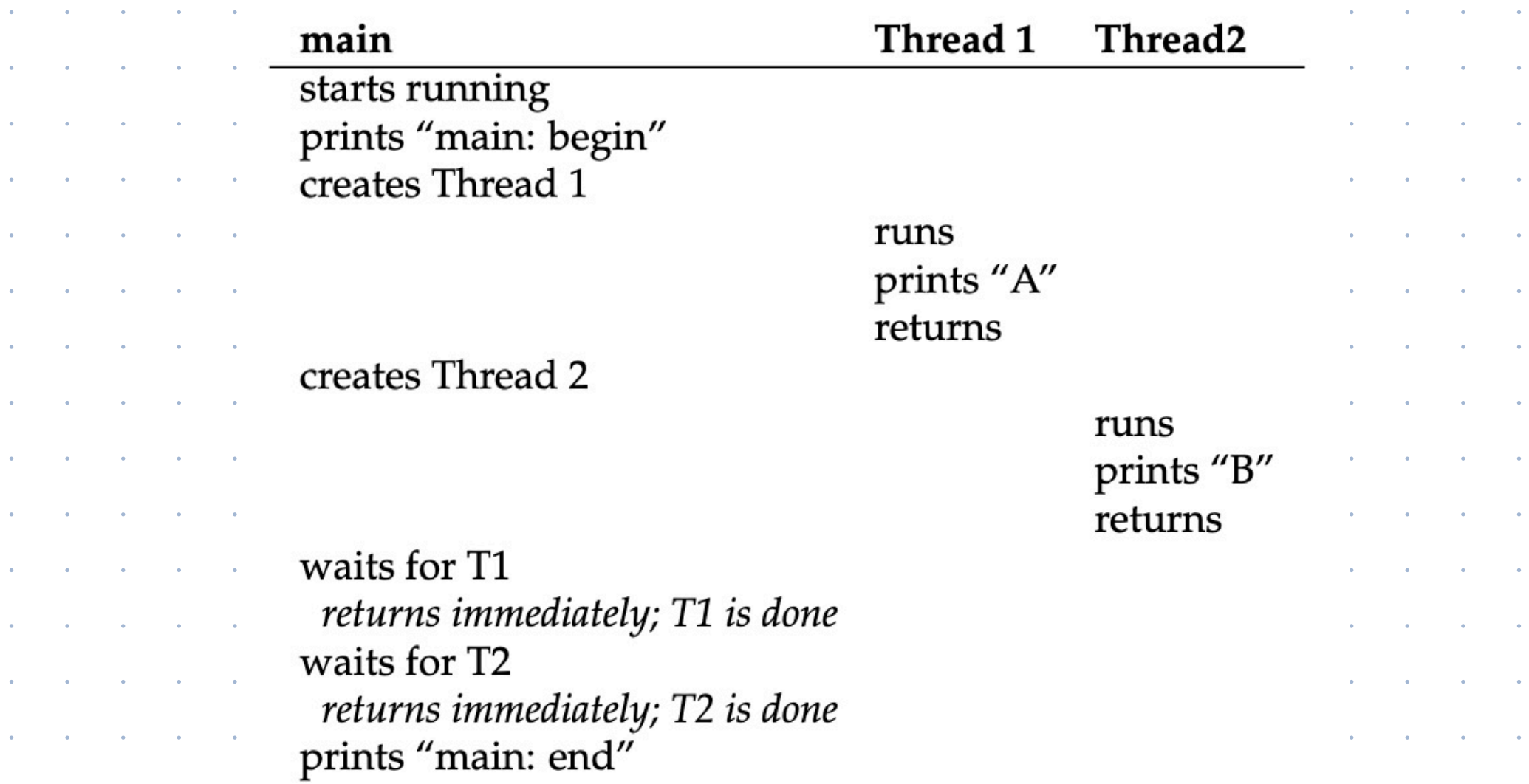


Figure 26.4: Thread Trace (2)

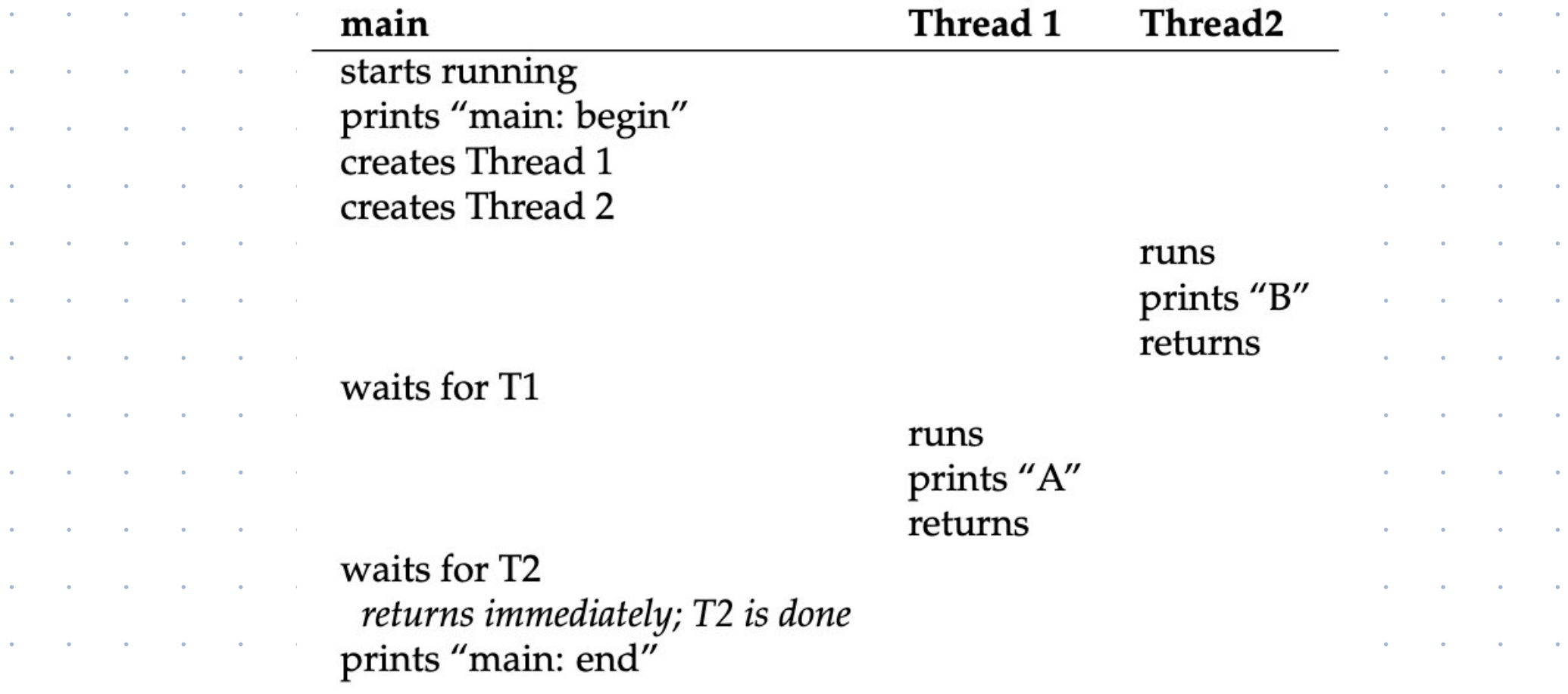


Figure 26.5: Thread Trace (3)

Why it Gets Noose: Shared Data

Let a simple program where two threads update a global shared variable.

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include "common.h"
4  #include "common_threads.h"
5
6  static volatile int counter = 0;
7
8  // mythread()
9  //
10 // Simply adds 1 to counter repeatedly, in a loop
11 // No, this is not how you would add 10,000,000 to
12 // a counter, but it shows the problem nicely.
13 //
14 void *mythread(void *arg) {
15     printf("%s: begin\n", (char *) arg);
16     int i;
17     for (i = 0; i < 1e7; i++) {
18         counter = counter + 1;
19     }
20     printf("%s: done\n", (char *) arg);
21     return NULL;
22 }
23
24 // main()
25 //
26 // Just launches two threads (pthread_create)
27 // and then waits for them (pthread_join)
28 //
29 int main(int argc, char *argv[]) {
30     pthread_t p1, p2;
31     printf("main: begin (counter = %d)\n", counter);
32     Pthread_create(&p1, NULL, mythread, "A");
33     Pthread_create(&p2, NULL, mythread, "B");
34
35     // join waits for the threads to finish
36     Pthread_join(p1, NULL);
37     Pthread_join(p2, NULL);
38     printf("main: done with both (counter = %d)\n",
39           counter);
40     return 0;
41 }
```

Figure 26.6: Sharing Data: Uh Oh (t1.c)

The desired result is 20,000,000 but the result is undeterministic.

The Heart of the Problem: Uncontrolled Scheduling

↳ A increment to counter looks like below in assembly,

```
1: mov 0x8049a1c, %eax
2: add $0x1, %eax
3: mov %eax, 0x8049a1c
```

Address of counter variable

1: Move value at address into register eax

2: Add 1 to it

3: contents of eax is stored back at address

Lets say T_1 enters this code & executes instructions (1: $eax=50$ & 2: $eax=51$) but then timer interrupt goes off.

So, OS saves state of T_1 to it's TCB.

The OS decides to run T_2 & it enters this code. As previous incremented wasn't saved to memory & as each thread has it's own set registers i.e. the registers are virtualized by context switch, this thread starts at previous value.

Lets say T_2 successfully incremented by running all 3 instructions (1: $eax=50$ 2: $eax=51$ 3: $counter=51$) before context switch.

Once T_1 comes back, it executes the last instruction (3: $counter=51$)

The code was run twice but counter was updated once.

OS	Thread 1	Thread 2	(after instruction)		
			PC	eax	counter
	<i>before critical section</i>		100	0	50
	mov 8049a1c,%eax		105	50	50
	add \$0x1,%eax		108	51	50
interrupt	<i>save T1</i>				
	<i>restore T2</i>		100	0	50
		mov 8049a1c,%eax	105	50	50
		add \$0x1,%eax	108	51	50
		mov %eax,8049a1c	113	51	51
interrupt	<i>save T2</i>				
	<i>restore T1</i>		108	51	51
	mov %eax,8049a1c		113	51	51

Figure 26.7: The Problem: Up Close and Personal

The above code is at address 100d mov instruction takes 5 bytes.

This is called race condition or data race; the results depend on the timing of the code's execution.

What we have demonstrated here is called a **race condition** (or, more specifically, a **data race**): the results depend on the timing of the code's execution. With some bad luck (i.e., context switches that occur at untimely points in the execution), we get the wrong result. In fact, we may get a different result each time; thus, instead of a nice **deterministic** computation (which we are used to from computers), we call this result **indeterminate**, where it is not known what the output will be and it is indeed likely to be different across runs.

Because multiple threads executing this code can result in a race condition, we call this code a **critical section**. A critical section is a piece of code that accesses a shared variable (or more generally, a shared resource) and must not be concurrently executed by more than one thread.

What we really want for this code is what we call **mutual exclusion**. This property guarantees that if one thread is executing within the critical section, the others will be prevented from doing so.

The Wish For Atomicity

memory-add 0x8049a1c, \$0x1

(A super instruction)

Assume this instruction adds a value to a memory location, and hardware guarantees that it executes atomically.

When an interrupt occurs, either the instruction has not run at all or runs to completion; there is no in-between state.

So, naively, we would have such super instructions. Issue them & leave it to the hardware but do we want the hardware to support an "atomic update of B-tree" instruction?

Instead, we will use synchronization primitives

ASIDE: KEY CONCURRENCY TERMS CRITICAL SECTION, RACE CONDITION, INDETERMINATE, MUTUAL EXCLUSION

These four terms are so central to concurrent code that we thought it worth while to call them out explicitly. See some of Dijkstra's early work [D65,D68] for more details.

- A **critical section** is a piece of code that accesses a *shared* resource, usually a variable or data structure.
- A **race condition** (or **data race** [NM92]) arises if multiple threads of execution enter the critical section at roughly the same time; both attempt to update the shared data structure, leading to a surprising (and perhaps undesirable) outcome.
- An **indeterminate** program consists of one or more race conditions; the output of the program varies from run to run, depending on which threads ran when. The outcome is thus not **deterministic**, something we usually expect from computer systems.
- To avoid these problems, threads should use some kind of **mutual exclusion** primitives; doing so guarantees that only a single thread ever enters a critical section, thus avoiding races, and resulting in deterministic program outputs.